Sprint 19 Structured Notes: Regular Expressions from Basics to Advanced Patterns

0. What Sprint 19 Is About

Sprint 19 is about **regular expressions**, usually called **regex**.

A regex is a pattern used to search, validate, extract, and replace text.

The main goal of Sprint 19 is:

> Search, validate, extract, and replace text with patterns.

Before this sprint, you already used string tools like:

```
includes()
indexOf()
slice()
trim()
toUpperCase()
toLowerCase()
replace()
```

Those tools are useful when you know the exact text you want.

Regex is useful when you do **not** only want exact text. Regex is useful when you want a **pattern**.

Example:

```
"cat"
```

This is exact text.

But this idea:

```
any 4-digit number
```

is a pattern.

In JavaScript, that pattern can be written as:

```
/\d{4}/
```

Simple meaning:

```
Find 4 digits in a row.
```

---

1. Current Progress Before Sprint 19

Sprint 19 builds on earlier sprints.

Sprint 3 connection: Strings

Sprint 3 taught you how to work with text.

You learned:

- strings are text
- strings are immutable
- string methods return new strings
- `includes()` checks if text exists
- `indexOf()` finds a position
- `replace()` replaces text
- `slice()` extracts text

Sprint 19 extends this.

Instead of searching only for exact text, you search for text patterns.

Example with exact text:

```
const text = "My code is 1234";
console.log(text.includes("1234"));
```

Output:

```
true
```

Example with regex pattern:

```
const text = "My code is 1234";
console.log(/\d{4}/.test(text));
```

Output:

```
true
```

The regex version does not only search for `1234`. It searches for **any 4 digits**.

Sprint 15 connection: Forms and validation

Sprint 15 taught you form validation.

You learned:

```
<input required>
<input type="email">
<input minlength="10">
<input maxlength="100">
<input pattern="...">
```

The `pattern` attribute uses regex-like rules.

Example:

```
<input pattern="[0-9]{4}">
```

Simple meaning:

```
The user must enter exactly 4 digits.
```

Valid:

```
1234
```

Invalid:

```
abcd
12
12345
```

Important beginner rule:

> Use built-in HTML validation first. Use regex only when you need a custom pattern.

Sprint 18 connection: Debugging

Sprint 18 taught you to debug systematically.

This matters because regex can become hard to read.

When a regex does not work, do not guess randomly.

Ask:

1. What pattern did I mean to write?
2. What text am I testing?
3. Did I forget `^` or `$`?
4. Did I use the `g` flag with `test()`?
5. Did I escape special characters correctly?
6. Did I accidentally match too much or too little?

---

2. What Is a Regular Expression?

A **regular expression** is a text pattern.

Simple meaning:

> Regex is a way to describe what kind of text you are looking for.

Example patterns:

```
Find the word cat.
Find any digit.
Find exactly 4 digits.
Find text that starts with A.
Find an email-like value.
```

```
Find repeated words.
Find a username with only letters, numbers, and underscores.
```

Regex is useful for:

- searching text
- checking form input
- extracting useful parts of text
- replacing text
- cleaning text
- finding repeated patterns

---

3. Regex Literal Syntax

A regex literal is written between forward slashes.

```
/pattern/
```

Example:

```
/freeCodeCamp/
```

This pattern searches for the exact text:

```
freeCodeCamp
```

Example:

```
const regex = /JavaScript/;

console.log(regex.test("I am learning JavaScript"));
console.log(regex.test("I am learning Python"));
```

Output:

```
true
false
```

Simple reading:

```
Does this string contain JavaScript?
```

---

4. Regex Literal vs Normal String

This is a string:

```
"cat"
```

This is a regex:

```
/cat/
```

They look similar, but they are different values.

A string stores text.

A regex stores a search pattern.

Example:

```
const word = "cat";
const pattern = /cat/;

console.log(typeof word);
console.log(typeof pattern);
```

Output:

```
string
object
```

Beginner meaning:

> Regex values are special objects used for pattern matching.

---

5. `test()`

`test()` checks whether a string matches a regex pattern.

It returns a boolean:

```
true or false
```

Syntax:

```
regex.test(string)
```

Example:

```
const regex = /cat/;

console.log(regex.test("The cat is sleeping."));
console.log(regex.test("The dog is sleeping."));
```

Output:

```
true
false
```

Simple meaning:

```
Does the string contain cat?
```

Beginner rule:

> Use `test()` when you only need a yes/no answer.

---

6. `test()` for Validation

Validation means checking whether input follows rules.

Example:

```
const usernameRegex = /^[A-Za-z0-9_]+$/;

console.log(usernameRegex.test("sangeeth_123"));
console.log(usernameRegex.test("sangeeth-123"));
```

Output:

```
true
false
```

Why is the second one false?

Because the pattern allows:

```
letters
numbers
underscore
```

It does not allow:

```
hyphen
```

---

7. `match()`

`match()` searches a string and returns details about the match.

Syntax:

```
string.match(regex)
```

Example:

```
const text = "My code is 1234";
const result = text.match(/\d{4}/);

console.log(result);
```

Conceptual output:

```
["1234"]
```

Simple meaning:

```
Find the first 4-digit number and return information about it.
```

Beginner rule:

```
test()  -> Did it match?
match() -> What matched?
```

---

8. When `match()` Finds Nothing

If `match()` does not find anything, it returns `null`.

Example:

```
const result = "No code here".match(/\d{4}/);
console.log(result);
```

Output:

```
null
```

Simple meaning:

```
No match was found.
```

Important:

Before using a match result, check whether it is `null`.

Safer code:

```
const text = "My code is 1234";
const result = text.match(/\d{4}/);
```

```
if (result !== null) {
  console.log(result[0]);
} else {
  console.log("No code found.");
}
```

---

9. `replace()`

`replace()` returns a new string with matched text replaced.

Syntax:

```
string.replace(pattern, replacement)
```

Example:

```
const text = "My phone code is 1234";
const updated = text.replace(/\d{4}/, "****");

console.log(updated);
```

Output:

```
My phone code is ****
```

Simple meaning:

```
Find 4 digits and replace them with ****.
```

Important:

`replace()` does not change the original string.

Example:

```
const text = "cat dog";
const updated = text.replace(/cat/, "bird");
```

```
console.log(text);
console.log(updated);
```

Output:

```
cat dog
bird dog
```

Why?

Because strings are immutable. The method returns a new string.

---

10. `replace()` Replaces the First Match by Default

Without global matching, `replace()` replaces only the first match.

Example:

```
const text = "cat cat cat";
const updated = text.replace(/cat/, "dog");

console.log(updated);
```

Output:

```
dog cat cat
```

Only the first `cat` changed.

To replace all matches, use the `g` flag:

```
const text = "cat cat cat";
const updated = text.replace(/cat/g, "dog");

console.log(updated);
```

Output:

```
dog dog dog
```

---

11. `replaceAll()`

`replaceAll()` replaces every occurrence.

Example with a string:

```
const text = "freecodecamp is great. freecodecamp helps.";
const updated = text.replaceAll("freecodecamp", "freeCodeCamp");

console.log(updated);
```

Output:

```
freeCodeCamp is great. freeCodeCamp helps.
```

When using `replaceAll()` with a regex, the regex must be global.

Correct:

```
const text = "cat cat cat";
console.log(text.replaceAll(/cat/g, "dog"));
```

Output:

```
dog dog dog
```

Incorrect:

```
const text = "cat cat cat";
console.log(text.replaceAll(/cat/, "dog"));
```

This causes an error because the regex does not use `g`.

Beginner rule:

Use `replace()` for one replacement. Use `replace()` with `g` or `replaceAll()` for all replacements.

---

12. Exact Character Matching

A regex can match exact characters.

Example:

```
/cat/
```

This matches the exact letters:

```
c a t
```

Example:

```
console.log(/cat/.test("cat"));
console.log(/cat/.test("category"));
console.log(/cat/.test("dog"));
```

Output:

```
true
true
false
```

Why does `category` return true?

Because it contains:

```
cat
```

Important:

```
/cat/
```

means:

```
Find cat anywhere in the string.
```

It does not mean:

```
The whole string must be exactly cat.
```

For that, use anchors.

---

13. Anchors: `^` and `$`

Anchors control position.

```
^
```

means start of string.

```
$
```

means end of string.

Example:

```
const regex = /^cat$/;

console.log(regex.test("cat"));
console.log(regex.test("category"));
console.log(regex.test("my cat"));
```

Output:

```
true
false
false
```

Simple meaning:

```
The whole string must be exactly cat.
```

Comparison:

```
/cat/
```

means:

```
contains cat somewhere
```

```
/^cat$/
```

means:

```
starts with cat and ends immediately after cat
```

Beginner rule:

> Use `^` and `$` when validating the whole input.

---

14. Multiline Anchors with `m`

Normally, `^` and `$` work with the whole string.

The `m` flag lets them work with each line inside a multiline string.

Example:

```
const text = `cat
dog
cat`;

console.log(text.match(/^cat/g));
```

Output:

```
["cat"]
```

Only the first line starts at the beginning of the whole string.

With `m`:

```
const text = `cat
dog
cat`;

console.log(text.match(/^cat/gm));
```

Output:

```
["cat", "cat"]
```

Simple meaning:

```
With m, ^ can match the start of each line.
```

---

15. The Dot: `.`

The dot means:

```
any single character except a line break
```

Example:

```
console.log(/c.t/.test("cat"));
console.log(/c.t/.test("cut"));
console.log(/c.t/.test("cot"));
console.log(/c.t/.test("ct"));
```

Output:

```
true
true
true
false
```

Why is the last one false?

Because `.` needs one character between `c` and `t`.

---

16. Dot with the `s` Flag

The `s` flag lets dot match line breaks too.

Without `s`:

```
const text = "a\nb";
console.log(/a.b/.test(text));
```

Output:

```
false
```

With `s`:

```
const text = "a\nb";
console.log(/a.b/s.test(text));
```

Output:

```
true
```

Simple meaning:

```
s lets . match across line breaks.
```

---

17. Character Classes

A character class means:

```
match one character from this group
```

Syntax:

```
/[abc]/
```

Simple meaning:

```
Match a, b, or c.
```

Example:

```
console.log(/[abc]/.test("dog"));
console.log(/[abc]/.test("bat"));
```

Output:

```
false
true
```

Why does `bat` return true?

Because it contains `b` and `a`, and either one is enough for this pattern.

---

18. Character Ranges

Ranges let you write a group more compactly.

```
/[a-z]/
```

means any lowercase letter.

```
/[A-Z]/
```

means any uppercase letter.

```
/[0-9]/
```

means any digit.

Example:

```
console.log(/[a-z]/.test("ABC"));
console.log(/[a-z]/.test("ABc"));
console.log(/[0-9]/.test("abc5"));
```

Output:

```
false
true
true
```

---

19. Combining Ranges

You can combine ranges inside one character class.

Example:

```
/[a-zA-Z0-9]/
```

Simple meaning:

```
one lowercase letter, uppercase letter, or digit
```

Example:

```
const regex = /^[a-zA-Z0-9]+$/;

console.log(regex.test("abc123"));
console.log(regex.test("ABC123"));
console.log(regex.test("abc_123"));
```

Output:

```
true
true
false
```

The underscore fails because the pattern did not include `_`.

---

20. Matching a Literal Hyphen in a Character Class

Inside a character class, `-` often means a range.

Example:

```
/[a-z]/
```

means lowercase letters from `a` to `z`.

If you want to match a real hyphen, put it at the beginning or end of the class.

Example:

```
/[-a-z]/
```

or:

```
/[a-z-]/
```

Both can match lowercase letters or a literal hyphen.

Example:

```
const regex = /^[a-z-]+$/;

console.log(regex.test("hello-world"));
console.log(regex.test("hello_world"));
```

Output:

```
true
false
```

---

21. Negated Character Classes

A caret inside a character class can mean "not".

```
/[^0-9]/
```

Simple meaning:

```
match one character that is not a digit
```

Example:

```
console.log(/[^0-9]/.test("123"));
console.log(/[^0-9]/.test("123a"));
```

Output:

```
false
true
```

Important:

`^` means different things depending on where it appears.

At the start of the regex:

```
/^cat/
```

means:

```
starts with cat
```

Inside a character class:

```
/[^cat]/
```

means:

```
one character that is not c, a, or t
```

---

22. Shortcut Character Classes

JavaScript has shortcut classes for common patterns.

```
\d
```

means digit.

Same idea as:

```
[0-9]
```

```
\w
```

means word character.

Usually:

```
letter, digit, or underscore
```

```
\s
```

means whitespace.

This includes spaces, tabs, and line breaks.

Example:

```
console.log(/\d/.test("abc123"));
console.log(/\w/.test("___"));
console.log(/\s/.test("a b"));
```

Output:

```
true
true
true
```

---

23. Uppercase Shortcut Classes

Uppercase shortcuts mean the opposite.

```
\D
```

means not a digit.

```
\W
```

means not a word character.

```
\S
```

means not whitespace.

Example:

```
console.log(/\D/.test("123"));
console.log(/\D/.test("123a"));
console.log(/\S/.test("   "));
console.log(/\S/.test(" a "));
```

Output:

```
false
true
false
true
```

---

24. Quantifiers

Quantifiers control how many times the previous pattern item can appear.

Common quantifiers:

```
+      one or more
*      zero or more
```

```
?      zero or one
{4}    exactly 4
{4,}   4 or more
{4,6}  between 4 and 6
```

---

25. `+`: One or More

```
/\d+/
```

Simple meaning:

```
one or more digits
```

Example:

```
console.log(/\d+/.test("abc123"));
console.log(/\d+/.test("abc"));
```

Output:

```
true
false
```

---

26. `*`: Zero or More

```
/ab*c/
```

Simple meaning:

```
a, then zero or more b characters, then c
```

Example:

```
console.log(/ab*c/.test("ac"));
console.log(/ab*c/.test("abc"));
console.log(/ab*c/.test("abbbc"));
```

Output:

```
true
true
true
```

Why does `ac` pass?

Because `b*` allows zero `b` characters.

---

27. `?` : Zero or One

```
/colou?r/
```

Simple meaning:

```
u is optional
```

Example:

```
console.log(/colou?r/.test("color"));
console.log(/colou?r/.test("colour"));
console.log(/colou?r/.test("colouur"));
```

Output:

```
true
true
false
```

---

28. `{n}`: Exact Count

```
/\d{4}/
```

Simple meaning:

```
exactly 4 digits in a row
```

Example:

```
console.log(/\d{4}/.test("1234"));
console.log(/\d{4}/.test("123"));
console.log(/\d{4}/.test("Code 12345"));
```

Output:

```
true
false
true
```

Why does `Code 12345` return true?

Because it contains at least one section of 4 digits: `1234`.

If the whole string must be exactly 4 digits, use anchors:

```
/^\d{4}$/
```

Example:

```
console.log(/^\d{4}$/.test("1234"));
console.log(/^\d{4}$/.test("12345"));
```

Output:

```
true
false
```

---

29. `{n,}`: At Least Count

```
/\d{4,}/
```

Simple meaning:

```
4 or more digits
```

Example:

```
console.log(/^\d{4,}$/.test("123"));
console.log(/^\d{4,}$/.test("1234"));
console.log(/^\d{4,}$/.test("12345"));
```

Output:

```
false
true
true
```

---

30. `{n,m}`: Range Count

```
/\d{4,6}/
```

Simple meaning:

```
between 4 and 6 digits
```

Example:

```
console.log(/^\d{4,6}$/.test("123"));
console.log(/^\d{4,6}$/.test("1234"));
console.log(/^\d{4,6}$/.test("123456"));
console.log(/^\d{4,6}$/.test("1234567"));
```

Output:

```
false
true
true
false
```

---

31. Flags

Flags change how a regex behaves.

Flags are written after the closing slash.

```
/pattern/flags
```

Example:

```
/hello/i
```

The curriculum lists these flags:

```
i  ignore case
g  global matching
m  multiline anchors
d  match indices
u  Unicode-aware behavior
v  expanded Unicode matching
y  sticky matching
s  dotAll, dot matches line breaks
```

You do not need to master every advanced detail immediately. But you should know what each flag is for.

---

32. `i` Flag: Ignore Case

The `i` flag makes matching case-insensitive.

Example:

```
console.log(/hello/.test("HELLO"));
console.log(/hello/i.test("HELLO"));
```

Output:

```
false
true
```

Simple meaning:

```
Ignore uppercase/lowercase differences.
```

---

33. `g` Flag: Global Matching

The `g` flag finds all matches instead of only the first match.

Example:

```
const text = "cat cat cat";

console.log(text.match(/cat/));
console.log(text.match(/cat/g));
```

Conceptual output:

```
["cat"]
["cat", "cat", "cat"]
```

Simple meaning:

```
g means keep searching after the first match.
```

---

34. `g` Flag Warning: Stateful Regex and `lastIndex`

The `g` flag can make a regex stateful.

Stateful means:

```
The regex remembers where it stopped last time.
```

That remembered position is stored in:

```
regex.lastIndex
```

Example:

```
const regex = /cat/g;

console.log(regex.test("cat"));
console.log(regex.test("cat"));
console.log(regex.test("cat"));
```

Output:

```
true
false
true
```

This surprises many beginners.

Why does this happen?

After the first `true`, the regex remembers that it matched at the end of the string. The next test starts searching from that remembered position, so it does not find another `cat`. Then JavaScript resets and the next call can return true again.

Safer beginner rule:

> Do not use `g` with `test()` when you are checking many independent strings, unless you reset `lastIndex`.

Better:

```
console.log(/cat/.test("cat"));
console.log(/cat/.test("cat"));
console.log(/cat/.test("cat"));
```

Output:

```
true
true
true
```

If you must reuse a global regex, reset it:

```
const regex = /cat/g;

regex.lastIndex = 0;
console.log(regex.test("cat"));

regex.lastIndex = 0;
console.log(regex.test("cat"));
```

---

35. `m` Flag: Multiline

The `m` flag changes how `^` and `$` behave.

Without `m`, they match the start and end of the whole string.

With `m`, they can match the start and end of each line.

Example:

```
const text = `start
middle
start`;

console.log(text.match(/^start/g));
console.log(text.match(/^start/gm));
```

Conceptual output:

```
["start"]
["start", "start"]
```

---

36. `s` Flag: DotAll

The `s` flag lets `.` match line breaks.

Example:

```
const text = "a\nb";

console.log(/a.b/.test(text));
console.log(/a.b/s.test(text));
```

Output:

```
false
true
```

---

37. `u` Flag: Unicode-Aware Behavior

The `u` flag makes regex handle Unicode patterns more correctly.

Beginner meaning:

> Use `u` when you are working with Unicode characters and want stricter, more modern Unicode behavior.

Example:

```
const smile = "😀";

console.log(/^.$/.test(smile));
console.log(/^.$/u.test(smile));
```

Possible output:

```
false
true
```

Explanation:

Some characters, like emoji, are stored internally in a way that can confuse simple character matching. The `u` flag helps regex treat them more like one character.

---

38. `v` Flag: Expanded Unicode Matching

The `v` flag is an advanced Unicode flag.

Beginner meaning:

> The `v` flag expands Unicode matching features beyond `u`.

You only need awareness at this stage.

Use it later when you are specifically working with advanced Unicode sets and modern regex features.

---

39. `y` Flag: Sticky Matching

The `y` flag means sticky matching.

Sticky means:

```
The regex must match exactly at lastIndex.
```

Example:

```
const text = "cat dog";
const regex = /dog/y;

regex.lastIndex = 4;
console.log(regex.test(text));
```

Output:

```
true
```

Why?

Index `4` is where `dog` begins.

If `lastIndex` is wrong:

```
const text = "cat dog";
const regex = /dog/y;

regex.lastIndex = 0;
console.log(regex.test(text));
```

Output:

```
false
```

Beginner meaning:

> `y` is stricter than `g`. It does not search ahead. It only tries to match at the exact current position.

---

40. `d` Flag: Match Indices

The `d` flag adds index information to regex results.

Beginner meaning:

> It can tell you where the match starts and ends.

Example:

```
const text = "Code 1234";
const result = /\d+/d.exec(text);

console.log(result.indices[0]);
```

Conceptual output:

```
[5, 9]
```

Simple meaning:

```
The match starts at index 5 and ends before index 9.
```

Beginner warning:

`d` is useful, but more advanced. You can usually start with `match()`, `matchAll()`, and normal indexes.

---

41. `matchAll()`

`matchAll()` returns an iterator of all detailed match results.

Use it when you need:

- all matches
- capture groups
- match positions
- detailed match arrays

Important:

When you pass a regex to `matchAll()`, it must use the `g` flag.

Example:

```
const text = "Email: ada@example.com, backup: grace@example.com";
const emailRegex = /([a-z]+)@example\.com/gi;

for (const match of text.matchAll(emailRegex)) {
  console.log(match[0]);
  console.log(match[1]);
}
```

Output:

```
ada@example.com
ada
grace@example.com
grace
```

Explanation:

```
match[0]
```

is the full match.

```
match[1]
```

is the first captured group.

---

42. `matchAll()` Returns an Iterator

`matchAll()` does not directly return a normal array.

It returns an iterator.

Simple meaning:

```
It gives results one at a time.
```

You can loop through it:

```
for (const match of text.matchAll(regex)) {
  console.log(match);
}
```

Or convert it into an array:

```
const matches = Array.from(text.matchAll(regex));
```

Use this when you want to store all match results.

Example:

```
const text = "A1 B2 C3";
const regex = /([A-Z])(\d)/g;

const matches = Array.from(text.matchAll(regex));

console.log(matches.length);
console.log(matches[0][0]);
console.log(matches[0][1]);
console.log(matches[0][2]);
```

Output:

```
3
A1
A
1
```

---

43. Capturing Groups

Parentheses capture part of a match.

Syntax:

```
(pattern)
```

Example:

```
const regex = /([a-z]+)@example\.com/i;
const result = "ada@example.com".match(regex);

console.log(result[0]);
console.log(result[1]);
```

Output:

```
ada@example.com
ada
```

Explanation:

```
result[0]
```

is the full match.

```
result[1]
```

is the first captured group.

This group:

```
([a-z]+)
```

captures the username part before `@example.com`.

---

44. Multiple Capturing Groups

You can capture multiple parts.

Example:

```
const regex = /(\d{4})-(\d{2})-(\d{2})/;
const result = "Date: 2026-05-20".match(regex);

console.log(result[0]);
console.log(result[1]);
console.log(result[2]);
console.log(result[3]);
```

Output:

```
2026-05-20
2026
05
20
```

Simple meaning:

```
Group 1 captured the year.
Group 2 captured the month.
Group 3 captured the day.
```

---

45. Named Capturing Groups

Named groups let you give a group a name.

Syntax:

```
(?<name>pattern)
```

Example:

```
const regex = /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/;
const result = "2026-05-20".match(regex);

console.log(result.groups.year);
console.log(result.groups.month);
console.log(result.groups.day);
```

Output:

```
2026
05
20
```

Simple meaning:

> Named groups make match results easier to read.

---

46. Non-Capturing Groups

A non-capturing group groups a pattern without saving it as a result.

Syntax:

```
(?:pattern)
```

Example:

```
const regex = /(?:Mr|Ms)\. ([A-Z][a-z]+)/;
const result = "Ms. Ada".match(regex);

console.log(result[0]);
console.log(result[1]);
```

Output:

```
Ms. Ada
Ada
```

Explanation:

```
(?:Mr|Ms)
```

groups `Mr` or `Ms`, but does not capture it.

```
([A-Z][a-z]+)
```

captures the name.

Beginner rule:

> Use `(?:...)` when you need grouping but do not need that group in the result.

---

47. Alternation: `|`

The pipe symbol means OR.

Example:

```
/cat|dog/
```

Simple meaning:

```
match cat or dog
```

Example:

```
console.log(/cat|dog/.test("I have a cat"));
console.log(/cat|dog/.test("I have a dog"));
console.log(/cat|dog/.test("I have a bird"));
```

Output:

```
true
true
false
```

With grouping:

```
/(cat|dog)s?/
```

This can match:

```
cat
cats
dog
dogs
```

---

48. Escaping Special Characters

Some regex characters have special meaning.

Examples:

```
. ^ $ * + ? ( ) [ ] { } |
```

If you want to match the real character, escape it with a backslash.

Example:

```
const regex = /example\.com/;

console.log(regex.test("example.com"));
console.log(regex.test("exampleXcom"));
```

Output:

```
true
false
```

Why?

```
\.
```

means a real period.

But:

```
.
```

means any single character except a line break.

---

49. Escaping Backslashes in Strings

Regex literals and string-based regexes are different.

Regex literal:

```
/\d+/
```

String passed to `RegExp`:

```
new RegExp("\\d+")
```

Why two backslashes?

Because strings also use backslashes for escapes.

Beginner rule:

> Regex literals are usually easier for beginners than `new RegExp()`.

Use `new RegExp()` mainly when you need to build a regex from a variable.

Example:

```
const word = "cat";
const regex = new RegExp(word, "i");

console.log(regex.test("CAT"));
```

Output:

```
true
```

---

50. Backreferences Inside Regex

A backreference reuses captured text inside the regex.

Numbered backreference:

```
\1
```

This means:

```
match the same text captured by group 1
```

Example:

```
const regex = /\b(\w+)\s+\1\b/i;

console.log(regex.test("the the"));
console.log(regex.test("the cat"));
```

Output:

```
true
false
```

Explanation:

```
(\w+)
```

captures a word.

```
\s+
```

matches spaces.

```
\1
```

matches the same word again.

---

51. Backreferences in Replacement Strings

In replacement strings, use:

```
$1
$2
$3
```

for numbered captured groups.

Example:

```
const text = "2026-05-20";
const updated = text.replace(/(\d{4})-(\d{2})-(\d{2})/, "$2/$3/$1");

console.log(updated);
```

Output:

```
05/20/2026
```

Explanation:

```
$1 = 2026
$2 = 05
$3 = 20
```

Replacement string:

```
$2/$3/$1
```

becomes:

```
05/20/2026
```

---

52. Named Backreferences

Named groups can be reused by name.

Inside regex:

```
\k<name>
```

In replacement strings:

```
$<name>
```

Example inside regex:

```
const regex = /\b(?<word>\w+)\s+\k<word>\b/i;

console.log(regex.test("hello hello"));
console.log(regex.test("hello world"));
```

Output:

```
true
false
```

Example in replacement:

```
const text = "2026-05-20";
const regex = /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/;

const updated = text.replace(regex, "$<month>/$<day>/$<year>");

console.log(updated);
```

Output:

```
05/20/2026
```

---

53. Lookahead

Lookahead checks what comes after the current position without including it in the match.

Positive lookahead:

```
(?=pattern)
```

Simple meaning:

```
Only match here if this pattern comes next.
```

Example:

```
const text = "ada@example.com grace@test.com";
const regex = /\w+(?=@example\.com)/g;

console.log(text.match(regex));
```

Output:

```
["ada"]
```

Why?

The regex matches word characters only if they are followed by:

```
@example.com
```

The `@example.com` part is checked but not included in the final match.

---

54. Negative Lookahead

Negative lookahead checks that something does **not** come next.

Syntax:

```
(?!pattern)
```

Simple meaning:

```
Only match here if this pattern does not come next.
```

Example:

```
const text = "file.txt file.exe file.md";
const regex = /file\.(?!exe)\w+/g;

console.log(text.match(regex));
```

Output:

```
["file.txt", "file.md"]
```

Why?

The regex skips `file.exe` because `(?!exe)` says:

```
Do not match if exe comes next.
```

---

55. Lookbehind

Lookbehind checks what comes before the current position without including it in the match.

Positive lookbehind:

```
(?<=pattern)
```

Simple meaning:

```
Only match here if this pattern came before.
```

Example:

```
const text = "Price: $25";
const regex = /(?<=\$)\d+/;

console.log(text.match(regex)[0]);
```

Output:

```
25
```

Why?

The regex matches digits only if they come after `$`.

The `$` is checked but not included in the final match.

---

56. Negative Lookbehind

Negative lookbehind checks that something did **not** come before.

Syntax:

```
(?<!pattern)
```

Simple meaning:

```
Only match here if this pattern did not come before.
```

Example:

```
const text = "$25 30 $40 50";
const regex = /(?<!\$)\b\d+\b/g;

console.log(text.match(regex));
```

Output:

```
["30", "50"]
```

Why?

The regex matches numbers that are not immediately preceded by `$`.

---

57. Word Boundaries: `\b`

A word boundary marks the edge of a word.

Syntax:

```
\b
```

Example:

```
console.log(/\bcat\b/.test("cat"));
console.log(/\bcat\b/.test("category"));
console.log(/\bcat\b/.test("my cat sleeps"));
```

Output:

```
true
false
true
```

Simple meaning:

```
Match cat as a whole word, not just part of another word.
```

---

58. Practical Pattern: Whole Username Validation

Goal:

```
Username must be 3 to 20 characters.
Only letters, numbers, and underscores are allowed.
```

Code:

```
function isValidUsername(username) {
  const regex = /^[A-Za-z0-9_]{3,20}$/;
  return regex.test(username);
}

console.log(isValidUsername("sangeeth"));
console.log(isValidUsername("sangeeth_123"));
console.log(isValidUsername("sa"));
console.log(isValidUsername("sangeeth-123"));
console.log(isValidUsername("sangeeth 123"));
```

Output:

```
true
true
false
false
false
```

Read the regex:

```
^                 start of input
[A-Za-z0-9_]      allowed characters
{3,20}            between 3 and 20 characters
$                 end of input
```

Why anchors matter:

```
/[A-Za-z0-9_]{3,20}/
```

could match part of a bad string.

```
/^[A-Za-z0-9_]{3,20}$/
```

checks the whole string.

---

59. Practical Pattern: 4-Digit Code Validation

Goal:

```
The input must be exactly 4 digits.
```

Code:

```
function isValidCode(code) {
  return /^\d{4}$/.test(code);
}

console.log(isValidCode("1234"));
console.log(isValidCode("123"));
console.log(isValidCode("12345"));
console.log(isValidCode("12a4"));
```

Output:

```
true
false
false
false
```

Pattern explanation:

```
^      start
\d     digit
{4}    exactly four times
$      end
```

---

60. Practical Pattern: Extract Email Usernames

Goal:

```
Find example.com email addresses and extract the username part.
```

Code:

```
const text = "Email: ada@example.com, backup: grace@example.com";
const emailRegex = /([a-z]+)@example\.com/gi;

for (const match of text.matchAll(emailRegex)) {
  console.log(match[0], match[1]);
}
```

Output:

```
ada@example.com ada
grace@example.com grace
```

Explanation:

```
([a-z]+)        capture one or more letters
@example\.com   match @example.com
flags gi        global and case-insensitive
```

---

61. Practical Pattern: Replace Repeated Words

Goal:

```
Replace repeated words like "the the" with one copy.
```

Code:

```
const text = "This is the the problem.";
const cleaned = text.replace(/\b(\w+)\s+\1\b/gi, "$1");

console.log(cleaned);
```

Output:

```
This is the problem.
```

Explanation:

```
\b       word boundary
(\w+)    capture a word
\s+      one or more spaces
\1       the same word again
\b       word boundary
gi       global and case-insensitive
$1       replace with the first captured word
```

---

62. Practical Pattern: Preserve Captured Text in Replacement

Code:

```
const capture = /free(co+de)camp/i;
console.log("freecooooodecamp".replace(capture, "paid$1world"));
```

Output:

```
paidcooooodeworld
```

Explanation:

```
(co+de)
```

captures:

```
cooooode
```

Then:

```
$1
```

reuses that captured text in the replacement.

So:

```
freecooooodecamp
```

becomes:

```
paidcooooodeworld
```

---

63. Core Code Pattern from Sprint 19

```
const regex = /^[a-zA-Z]+\d{4,6}$/;
console.log(regex.test("A1234")); // true

const text = "freecodecamp is the best. freecodecamp!";
console.log(text.replaceAll(/freecodecamp/g, "freeCodeCamp"));

const capture = /free(co+de)camp/i;
console.log("freecooooodecamp".replace(capture, "paid$1world"));
```

Step-by-step:

```
/^[a-zA-Z]+\d{4,6}$/
```

means:

```
^            start of string
[a-zA-Z]+    one or more letters
\d{4,6}      4 to 6 digits
$            end of string
```

So:

```
"A1234"
```

passes because it has:

```
one letter + four digits
```

---

64. Regex and Forms

Regex is useful in forms, but do not overuse it.

Better beginner approach:

Use HTML first:

```
<input type="email" required>
```

Then add simple JavaScript rules when needed:

```
email.value.endsWith("@company.com")
```

Use regex when you need a real pattern:

```
<input pattern="[0-9]{4}">
```

or:

```
/^\d{4}$/.test(code)
```

Beginner warning:

> Complicated regex can be hard to read, hard to debug, and still imperfect.

---

65. When Regex Is the Right Tool

Use regex when the rule is pattern-based.

Good regex use cases:

```
must be exactly 4 digits
must contain only letters and numbers
find all repeated words
extract dates from text
replace all matching words
find values before or after a symbol
```

Example:

```
/^\d{4}$/
```

is good for checking exactly 4 digits.

---

66. When Regex Is Not the Best Tool

Do not use regex when a simple string method is clearer.

Clearer:

```
email.endsWith("@company.com")
```

Less clear:

```
/@company\.com$/.test(email)
```

Both can work, but the first one is easier to read.

Beginner rule:

> If a simple string method clearly solves the problem, use the simple string method.

---

67. Common Beginner Mistakes

Mistake 1: Forgetting anchors during validation

Weak:

```
/\d{4}/.test("abc1234xyz")
```

Output:

```
true
```

Why?

Because the string contains 4 digits.

Stricter:

```
/^\d{4}$/.test("abc1234xyz")
```

Output:

```
false
```

Why?

Because the whole string is not exactly 4 digits.

Mistake 2: Using `g` with repeated `test()` checks

Problem:

```
const regex = /cat/g;

console.log(regex.test("cat"));
console.log(regex.test("cat"));
```

Output:

```
true
false
```

Fix:

```
console.log(/cat/.test("cat"));
console.log(/cat/.test("cat"));
```

or reset:

```
const regex = /cat/g;

regex.lastIndex = 0;
console.log(regex.test("cat"));
```

```
regex.lastIndex = 0;
console.log(regex.test("cat"));
```

Mistake 3: Forgetting to escape special characters

Weak:

```
/example.com/.test("exampleXcom")
```

Output:

```
true
```

Why?

Because `.` means any character.

Correct:

```
/example\.com/.test("exampleXcom")
```

Output:

```
false
```

Mistake 4: Assuming `match()` always returns an array

Problem:

```
const result = "abc".match(/\d+/);
console.log(result[0]);
```

This fails because `result` is `null`.

Safer:

```
const result = "abc".match(/\d+/);

if (result !== null) {
  console.log(result[0]);
} else {
  console.log("No number found.");
}
```

Mistake 5: Making regex too complex too early

Weak habit:

```
Trying to solve every text problem with a giant regex.
```

Better habit:

```
Use simple string methods first when they are clearer.
Use regex only when the rule is truly pattern-based.
```

---

68. Debugging Regex

When a regex fails, use this checklist.

Step 1: Write the rule in plain English

Example:

```
The username must be 3 to 20 characters and only contain letters, numbers, and
underscores.
```

Step 2: Build the regex in small parts

```
[A-Za-z0-9_]      allowed character
[A-Za-z0-9_]{3,20} allowed length
^[A-Za-z0-9_]{3,20}$ whole input
```

Step 3: Test good inputs

```
console.log(/^[A-Za-z0-9_]{3,20}$/.test("sangeeth"));
console.log(/^[A-Za-z0-9_]{3,20}$/.test("abc_123"));
```

Step 4: Test bad inputs

```
console.log(/^[A-Za-z0-9_]{3,20}$/.test("ab"));
console.log(/^[A-Za-z0-9_]{3,20}$/.test("abc-123"));
console.log(/^[A-Za-z0-9_]{3,20}$/.test("abc 123"));
```

Step 5: Use `console.assert()` for expected behavior

```
function isValidUsername(username) {
  return /^[A-Za-z0-9_]{3,20}$/.test(username);
}

console.assert(isValidUsername("abc") === true, "abc should be valid");
console.assert(isValidUsername("ab") === false, "ab should be too short");
console.assert(isValidUsername("abc-123") === false, "hyphen should not be
valid");
```

---

69. Warm-Up Drills

Practice these in the console.

Drill 1: `test()`

```
console.log(/cat/.test("category"));
console.log(/^cat$/.test("category"));
console.log(/\d{4}/.test("Code: 1234"));
console.log(/^\d{4}$/.test("Code: 1234"));
```

Expected output:

```
true
false
true
false
```

Drill 2: `match()`

```
const text = "Order numbers: 1234 and 5678";

console.log(text.match(/\d{4}/));
console.log(text.match(/\d{4}/g));
```

Conceptual output:

```
["1234"]
["1234", "5678"]
```

Drill 3: `replace()`

```
console.log("hello hello".replace(/hello/, "hi"));
console.log("hello hello".replace(/hello/g, "hi"));
```

Output:

```
hi hello
hi hi
```

Drill 4: `matchAll()`

```
const text = "A1 B2 C3";
const regex = /([A-Z])(\d)/g;

for (const match of text.matchAll(regex)) {
  console.log(match[0], match[1], match[2]);
}
```

Output:

```
A1 A 1
B2 B 2
C3 C 3
```

Drill 5: Lookaround

```
console.log("$25 30".match(/(?<=\$)\d+/));
console.log("file.txt file.exe".match(/file\.(?!exe)\w+/g));
```

Conceptual output:

```
["25"]
["file.txt"]
```

---

70. Mini-Build: Username Validator

Goal

Build a function that checks whether a username is valid.

Rules:

```
3 to 20 characters
letters allowed
numbers allowed
underscore allowed
spaces not allowed
hyphens not allowed
```

Code

```
function isValidUsername(username) {
  const regex = /^[A-Za-z0-9_]{3,20}$/;
  return regex.test(username);
}

const usernames = [
  "sam",
```

```
  "sangeeth_123",
  "sa",
  "sangeeth-123",
  "sangeeth 123",
  "this_username_is_way_too_long"
];

for (const username of usernames) {
  console.log(username, isValidUsername(username));
}
```

Expected behavior

```
sam true
sangeeth_123 true
sa false
sangeeth-123 false
sangeeth 123 false
this_username_is_way_too_long false
```

---

71. Extra Challenge: Remove Repeated Words

Goal

Replace repeated words like:

```
the the
hello hello
```

with one copy.

Code

```
function removeRepeatedWords(text) {
  return text.replace(/\b(\w+)\s+\1\b/gi, "$1");
}

console.log(removeRepeatedWords("This is the the problem."));
console.log(removeRepeatedWords("Hello hello world."));
```

Expected output

```
This is the problem.
Hello world.
```

---

72. Sprint 19 Memory Card

Main idea

```
Regex = pattern-based text search.
```

Main methods

```
regex.test(str)          // true or false
str.match(regex)         // match array or null
str.replace(regex, text) // new string with replacement
str.replaceAll(...)      // replace every occurrence
str.matchAll(regex)      // iterator of detailed matches
```

Main symbols

```
^       start
$       end
.       any character except line break
\d      digit
\D      not digit
\w      word character
\W      not word character
\s      whitespace
\S      not whitespace
[]      one character from a set
[^]     one character not from a set
+       one or more
*       zero or more
?       zero or one
{n}     exactly n
{n,}    n or more
{n,m}   between n and m
()      capturing group
(?:)    non-capturing group
(?<>)   named capturing group
```

```
|       or
\       escape special character
\b      word boundary
\1      numbered backreference
\k<>    named backreference
$1      captured group in replacement
$<>     named group in replacement
```

Main flags

```
i   ignore case
g   global matching
m   multiline anchors
d   match indices
u   Unicode-aware behavior
v   expanded Unicode matching
y   sticky matching
s   dotAll, dot matches line breaks
```

---

73. Revision Questions

Answer these without looking back.

1. What is a regular expression?
2. What is the difference between a string and a regex?
3. What does `test()` return?
4. When should you use `test()`?
5. What does `match()` return when it finds a match?
6. What does `match()` return when it finds nothing?
7. What does `replace()` return?
8. Does `replace()` change the original string?
9. What does the `g` flag do?
10. Why can the `g` flag make `test()` surprising?
11. What is `lastIndex`?
12. What does the `i` flag do?
13. What does the `m` flag do?
14. What does the `s` flag do?
15. What does the `d` flag add?
16. What does the `u` flag help with?
17. What is the `v` flag for at a high level?
18. What does the `y` flag do?
19. What do `^` and `$` mean?
20. Why are anchors important for validation?

21. What does `.` mean in regex?
22. What does `\d` mean?
23. What does `\D` mean?
24. What does `\w` mean?
25. What does `\s` mean?
26. What does `[abc]` mean?
27. What does `[a-z]` mean?
28. How do you include a literal hyphen inside a character class?
29. What does `[^0-9]` mean?
30. What does `+` mean?
31. What does `*` mean?
32. What does `?` mean?
33. What does `{4}` mean?
34. What does `{4,}` mean?
35. What does `{4,6}` mean?
36. What is a capturing group?
37. What is a named capturing group?
38. What is a non-capturing group?
39. What is a backreference?
40. What does `$1` mean in a replacement string?
41. What does `\1` mean inside a regex?
42. What is a lookahead?
43. What is a negative lookahead?
44. What is a lookbehind?
45. What is a negative lookbehind?
46. What does `matchAll()` return?
47. Why does `matchAll()` need a global regex?
48. When should you use `Array.from(str.matchAll(regex))`?
49. When should you avoid regex?
50. What is the full regex for a username that allows 3 to 20 letters, numbers, and underscores?

---

74. Answer Key

1. A regular expression is a pattern used to search, validate, extract, or replace text.
2. A string stores text. A regex stores a search pattern.
3. `test()` returns `true` or `false`.
4. Use `test()` when you only need a yes/no answer.
5. `match()` returns a match array with details.
6. It returns `null`.
7. `replace()` returns a new string.
8. No. Strings are immutable, so the original string is not changed.
9. The `g` flag searches globally, meaning it can find all matches.
10. With `test()`, a global regex remembers where it stopped, which can change the next result.
11. `lastIndex` is the position where a stateful regex will continue searching.
12. The `i` flag ignores case.

13. The `m` flag lets `^` and `$` match line starts and line ends.
14. The `s` flag lets `.` match line breaks.
15. The `d` flag adds indices showing where matches start and end.
16. The `u` flag helps with Unicode-aware matching.
17. The `v` flag expands Unicode matching features.
18. The `y` flag makes matching sticky at `lastIndex`.
19. `^` means start. `$` means end.
20. Anchors make sure the whole input follows the pattern.
21. `.` means any single character except a line break, unless `s` is used.
22. `\d` means digit.
23. `\D` means not a digit.
24. `\w` means word character.
25. `\s` means whitespace.
26. `[abc]` means one character: `a`, `b`, or `c`.
27. `[a-z]` means one lowercase letter.
28. Put the hyphen at the beginning or end: `[-a-z]` or `[a-z-]`.
29. `[^0-9]` means one character that is not a digit.
30. `+` means one or more.
31. `*` means zero or more.
32. `?` means zero or one.
33. `{4}` means exactly 4.
34. `{4,}` means 4 or more.
35. `{4,6}` means between 4 and 6.
36. A capturing group saves part of a match using parentheses.
37. A named capturing group saves part of a match under a name.
38. A non-capturing group groups without saving the group result.
39. A backreference reuses captured text.
40. `$1` means the first captured group in a replacement string.
41. `\1` means match the same text captured by group 1.
42. A lookahead checks what comes after without including it.
43. A negative lookahead checks that something does not come after.
44. A lookbehind checks what came before without including it.
45. A negative lookbehind checks that something did not come before.
46. `matchAll()` returns an iterator of detailed match arrays.
47. It needs `g` because it is designed to return all matches.
48. Use `Array.from(str.matchAll(regex))` when you want to store all detailed matches in an array.
49. Avoid regex when a simple string method is clearer.
50. `^[A-Za-z0-9_]{3,20}$`

---

75. Minimum Passing Standard

You are ready to move past Sprint 19 when you can explain this code without looking:

```
const regex = /^[A-Za-z0-9_]{3,20}$/;

console.log(regex.test("sangeeth_123"));
console.log(regex.test("sangeeth-123"));

const text = "Email: ada@example.com, backup: grace@example.com";
const emailRegex = /([a-z]+)@example\.com/gi;

console.log(text.match(emailRegex));

for (const match of text.matchAll(emailRegex)) {
  console.log(match[0], match[1]);
}

const cleaned = "This is the the problem.".replace(/\b(\w+)\s+\1\b/gi, "$1");
console.log(cleaned);
```

You should be able to say:

```
The username regex checks the whole string.
It allows only letters, numbers, and underscores.
It requires 3 to 20 characters.
The hyphen example fails because hyphen is not allowed.
The email regex finds example.com email addresses.
The capturing group extracts the username before @example.com.
match() gives matches.
matchAll() gives detailed matches with groups.
The repeated-word regex finds the same word twice in a row.
$1 keeps one copy of the captured word.
```

---

76. Final Sprint 19 Summary

Sprint 19 teaches pattern-based text processing.

The most important idea is:

> Regex is for text patterns, not only exact text.

The most important methods are:

```
test      -> yes/no
match     -> matched result or null
```

```
replace   -> new string with replacement
replaceAll -> replace every occurrence
matchAll  -> all detailed matches
```

The most important validation habit is:

> Use anchors when the whole input must match.

The most important debugging warning is:

> Be careful using `g` with repeated `test()` calls because of `lastIndex`.

The most important beginner judgment is:

> Use regex when the rule is a pattern. Use simple string methods when they are clearer.